Predicting Used Car Prices with Machine Learning: A Comparative Study of Models and Preprocessing Strategies

Yifan Wu (1618332) **Xinkai Zhang (1619423)** **Zhenyu Zhang (1255206)** **Yiming Zhang (1681020)**

Abstract

Used car price prediction is a practically important regression task that requires careful handling of heterogeneous tabular data. In this paper, we investigate the effectiveness of machine learning methods for predicting second-hand vehicle prices using the UK Used Car Dataset, comprising approximately 100,000 listings across 11 manufacturers. We address four research questions concerning prediction accuracy, model selection, the impact of preprocessing strategies, and feature importance. We compare five models spanning high-bias to low-bias: Linear Regression, Ridge Regression, K-Nearest Neighbours, Random Forest, and a Multi-Layer Perceptron. Our experiments demonstrate that ensemble tree-based methods and neural networks substantially outperform linear baselines (RMSE improvement of X%), that log-transforming the skewed price target and applying group-median imputation are the most impactful preprocessing choices, and that vehicle age and annual mileage are the dominant price predictors.

1 Introduction

The second-hand car market is one of the largest consumer goods markets globally, with millions of transactions per year. Accurately estimating a fair market price benefits both buyers—who risk overpaying—and sellers—who risk underpricing their vehicles. Traditional valuation relies on domain expertise and lookup tables, but these approaches fail to capture the complex, non-linear interactions among vehicle attributes such as brand, age, mileage, fuel type, and transmission. Machine learning offers a data-driven alternative that can model these interactions directly from historical listing data (Gegic et al., 2019).

This project investigates machine learning methods for used car price prediction using the UK Used Car Dataset (Desai, 2021), a publicly available collection of ∼108,000 listings scraped from a major UK automotive marketplace. The dataset provides a rich combination of numerical and categorical features, real-world missing values, and a heavily right-skewed continuous target (price in GBP), making it well-suited for exploring both modelling and preprocessing questions.

Research Questions. We address four specific research questions (RQs):

RQ1 To what extent can machine learning models accurately predict used car prices from available features?

RQ2 How do models spanning the bias–variance spectrum (linear models, instance-based, tree-based, and neural networks) compare in predictive performance and generalisation?

RQ3 How do different data preprocessing strategies—target transformation, missing value imputation, and feature scaling—affect model performance?

RQ4 Which features contribute most significantly to price prediction, and how can their importance be interpreted?

RQ1 establishes a baseline understanding of task difficulty. RQ2 provides a systematic model comparison linked to the bias–variance trade-off. RQ3 directly examines the downstream impact of preprocessing decisions, with particular relevance to distance-sensitive models such as KNN. RQ4 connects model behaviour to domain knowledge about the used car market.

The remainder of the paper is structured as follows: Section 2 reviews related work; Section 3 describes our dataset, preprocessing pipeline, models, and evaluation setup; Section 4 presents experimental results; Section 5 analyses findings; and Section 6 concludes.

2 Literature Review

Price prediction for used vehicles has attracted considerable attention in the machine learning literature. Pudaruth (2014) provided an early comparative study applying Naive Bayes, Decision Trees, and Neural Networks to a small Mauritian dataset, finding that tree-based methods outperform Naive Bayes due to the non-Gaussian nature of automotive features. Gegic et al. (2019) extended this to a larger European dataset using gradient-boosted trees and Support Vector Regression, demonstrating that ensemble methods consistently outperform linear baselines, with mileage and vehicle age emerging as the two most predictive features—a finding we replicate and contextualise in Section 4. Shinde and Kotecha (2018) proposed a KNN-based system for used car valuation, showing that KNN achieves competitive accuracy on datasets with strong local price clusters (e.g. vehicles of the same model and year), but degrades significantly without feature scaling—motivating our investigation of preprocessing effects in RQ3. Listiani (2009) applied SVR to German car listings, finding that RBF kernels outperform linear kernels, and that log-transforming the price target substantially improved RMSE.

Neural network approaches have gained traction with the availability of larger datasets. Goodfellow et al. (2016) provide the theoretical basis for multilayer perceptrons as universal approximators, supporting their use for capturing non-linear feature interactions. Breiman (2001) introduced Random Forests, which have become a strong baseline for tabular regression due to their robustness to irrelevant features and insensitivity to feature scale.

Our work contributes to this literature by: (1) using a substantially larger and more diverse dataset than prior studies; (2) conducting a controlled comparison across five model families; and (3) performing a systematic ablation of preprocessing strategies, which prior work treats as fixed.

3 Method

3.1 Dataset

We use the **UK Used Car Dataset** (Desai, 2021), a publicly available collection of used vehicle listings scraped from a major UK online automotive marketplace. The raw dataset comprises 108,540 listings distributed across 11 CSV files, one per manufacturer: Audi, BMW, Ford (two subsets), Mercedes (two subsets), Hyundai, Skoda, Toyota, Vauxhall, and Volkswagen. All files share a common nine-column schema: `model`, `year`, `price` (GBP), `transmission`, `mileage`, `fuelType`, `tax` (annual road tax in GBP), `mpg` (miles per gallon), and `engineSize` (litres). Two additional "unclean" files (Ford Focus, Mercedes C-Class) are included as raw scrape outputs with malformed price strings and split mileage columns; these are used to illustrate real-world data quality challenges but are not included in the main training corpus.

Properties. Table 1 summarises the key properties of the combined dataset after loading. The target variable `price` is continuous and exhibits severe right-skew (skewness $= 2.29$), with a median of GBP 14,698 and values ranging from GBP 450 to GBP 159,999. Two features, `tax` (13.1% missing) and `mpg` (8.6% missing), contain substantial proportions of missing values arising from incomplete scraping. The dataset contains 9 brands after merging the two Ford and two Mercedes subsets, and 195 distinct car models. Linear separability between price strata is low given the continuous target, but a moderate negative correlation with `mileage` ($r = -0.47$) and positive correlation with `year` ($r = 0.52$) provide a clear signal for regression.

Property	Value
Total listings (raw)	108,540
Listings after cleaning	∼100,000
Number of brands	9
Number of models	195
Numerical features	6 (`year`, `price`, `mileage`, `tax`, `mpg`, `engineSize`)
Categorical features	3 (`model`, `transmission`, `fuelType`)
Target variable	`price` (continuous, GBP)
Target skewness (raw)	2.29
Target skewness (log)	−0.13
Missing values (`tax`)	13.1%
Missing values (`mpg`)	8.6%

Table 1: Summary statistics of the UK Used Car Dataset before and after cleaning. Listing counts are approximate post-cleaning due to duplicate and outlier removal.

3.2 Data Preprocessing

All preprocessing is implemented in Python using `pandas` and `scikit-learn`, and is fully reproducible via Notebook 01 in the accompanying code repository. The pipeline proceeds through five stages.

Stage 1: Schema Standardisation. Each per-brand CSV is loaded and filtered to the nine standard columns, discarding any extraneous columns

(e.g. the `tax(GBP)` column present in the Hyundai file due to a scraping inconsistency). All files are concatenated into a single DataFrame and a `brand` column is appended.

Stage 2: Validity Filtering. Four categories of physically implausible values are identified and treated as missing rather than deleted, preserving sample size:

- `engineSize` $= 0$: no internal combustion vehicle has zero displacement; likely a data-entry error.
- `mileage` $= 0$: used car listings with zero recorded mileage are implausible placeholders.
- `price` $<$ GBP 100: below any realistic transaction threshold.
- `year` $\notin [1990, 2023]$: values of 1970 and 2060 represent obvious entry errors.

Stage 3: Duplicate Removal. Exact duplicate rows (identical across all nine columns) are removed.

Stage 4: Outlier Removal. We apply the IQR method with a conservative factor of $3\times$ to `price`, `mileage`, and `tax`. A row is removed if its value falls below $Q_1 - 3 \cdot \text{IQR}$ or above $Q_3 + 3 \cdot \text{IQR}$. This factor retains genuine high-value luxury cars and high-mileage fleet vehicles while discarding clear data errors. The resulting price bounds are approximately GBP [1,229, 46,440].

Stage 5: Category Consolidation. Five fuel type labels and four transmission labels in the raw data include rare entries (e.g. *Other*, *Electric* for a small fraction of the dataset). We retain only {*Petrol*, *Diesel*, *Hybrid*, *Electric*} and {*Manual*, *Automatic*, *Semi-Auto*}, mapping all other values to `NaN` for subsequent imputation.

3.3 Missing Value Handling

Table 2 shows the missing value profile after validity filtering. We use **group-median imputation** for all numerical columns and **within-brand mode imputation** for categorical columns. Group-median imputation is preferred over global median because feature distributions are strongly stratified by brand and fuel type: for example, diesel engines have systematically higher displacement and different MPG profiles than petrol engines.

Feature	Imputation Strategy	Missing %
`tax`	Median within (`year`, engine bin)	$\sim$13%
`mpg`	Median within (`fuelType`, engine bin)	$\sim$9%
`engineSize`	Median within (`brand`, `fuelType`)	$<$1%
`mileage`	Median within (`year`, `brand`)	$<$1%
`fuelType`	Mode within `brand`	$<$1%
`transmission`	Mode within `brand`	$<$1%

Table 2: Missing value rates (post-cleaning) and imputation strategies. Engine bin refers to a coarse discretisation of `engineSize` into five intervals.

3.4 Feature Engineering

Beyond the nine raw features, we construct four derived features motivated by domain knowledge of vehicle depreciation:

- **`car_age`** $= 2024 -$ `year`: vehicle depreciation is primarily a function of age rather than the registration year in isolation.
- **`mileage_per_year`** $=$ `mileage`$/($`car_age` $+ 1)$: normalises mileage by age, distinguishing a high-mileage new car from a low-mileage old one. A car driven 20,000 miles in one year signals different usage patterns than one driven 100,000 miles over ten years.
- **`is_luxury`** $\in \{0, 1\}$: binary indicator for premium brands (Audi, BMW, Mercedes), which command a price premium independent of other features.
- **`log_price`** $= \log(1 +$ `price`$)$: log-transformation of the target to reduce right-skew (skewness from 2.29 to -0.13). All models are trained to predict `log_price`; results are exponentiated for interpretation.

3.5 Categorical Encoding

We apply three encoding strategies tailored to the cardinality and ordinality of each categorical feature:

- **Label encoding** for `transmission` (3 levels): Manual $\rightarrow$ 0, Semi-Auto $\rightarrow$ 1, Automatic $\rightarrow$ 2. The ordering reflects increasing automation level, making this a reasonable ordinal treatment.
- **One-hot encoding (OHE)** for `fuelType` (4 levels): avoids imposing a false ordinal relationship. This produces four binary indicator columns (`fuel_Petrol`, `fuel_Diesel`, `fuel_Hybrid`, `fuel_Electric`).

- **Target encoding** for `brand` and `model`: each category is replaced by the mean `log_price` of that group. Target encoding is preferred over OHE for these high-cardinality features (9 brands, 195 models) because it produces a compact, informative representation without the curse of dimensionality. In the modelling notebooks, target encoding statistics are computed on the training fold only to prevent label leakage.

3.6 Feature Scaling

The final modelling feature set comprises 14 features (Table 3). We prepare two scaled versions for experiments related to RQ3:

- **StandardScaler**: zero mean, unit variance. Standard choice for gradient-based models and regularised linear models.
- **MinMaxScaler**: maps each feature to $[0, 1]$. Used to study the effect of bounded vs. standardised inputs on KNN and neural network performance.

Tree-based models (Random Forest) are scale-invariant and always trained on unscaled features.

Feature	Type	Source
`car_age`	Numerical	Engineered
`mileage`	Numerical	Raw
`mileage_per_year`	Numerical	Engineered
`tax`	Numerical	Raw
`mpg`	Numerical	Raw
`engineSize`	Numerical	Raw
`transmission_enc`	Ordinal	Label-encoded
`is_luxury`	Binary	Engineered
`brand_mean_price`	Numerical	Target-encoded
`model_mean_price`	Numerical	Target-encoded
`fuel_Diesel`	Binary	OHE
`fuel_Electric`	Binary	OHE
`fuel_Hybrid`	Binary	OHE
`fuel_Petrol`	Binary	OHE
Target: `log_price`	Numerical	Engineered

Table 3: Final modelling feature set (14 features) used as input to all models.

3.7 Validation Setup

To enable fair comparison across all models and preprocessing variants, we adopt a fixed train–development–test validation protocol in Notebook 02. The cleaned dataset is first split into an 80% training-development pool and a 20% held-out test set using a fixed random seed (49). The training-development pool is then split again into 75% training and 25% development data. This produces an effective 60%/20%/20% train–development–test split.

The development set is used for hyperparameter selection and model comparison, while the test set is reserved for final evaluation and is not used during model selection. Split row indices are exported to `split_indices.json` so that later evaluation notebooks can reuse exactly the same data partition.

For computational efficiency, hyperparameter tuning is performed in a balanced mode using up to 25,000 training rows and 6,000 development rows. After the best configuration for each model is selected, the final model is refitted on the complete training split and then evaluated on the full development set. All categorical encoding and numerical scaling are implemented inside `scikit-learn` pipelines so that imputation, one-hot encoding, and scaling are learned only from the fitting data.

3.8 Models and Hyperparameters

We evaluate five model families covering linear, instance-based, tree-based, ensemble, and neural approaches. This provides both interpretable baselines and more flexible non-linear models.

1. **Ridge Regression**: a linear regression model with L2 regularisation. We tune $\alpha \in \{0.1, 1.0, 10.0, 100.0\}$ and use standard scaling.
2. **K-Nearest Neighbours Regression (KNN)**: an instance-based distance model. We tune $k \in \{5, 10, 20\}$ and compare uniform versus distance weighting. KNN is trained with standard scaling because distance calculations are sensitive to feature scale.
3. **Decision Tree Regression**: a single non-linear tree model. We tune `max_depth` $\in \{8, 16, \texttt{None}\}$ and `min_samples_leaf` $\in \{1, 10, 50\}$. No scaling is applied because tree splits are scale-invariant.
4. **Random Forest Regression**: an ensemble of decision trees trained with bagging. We tune `n_estimators` $\in \{100, 200\}$, `max_depth` $\in \{16, \texttt{None}\}$, and `min_samples_leaf` $\in \{1, 10\}$. No scaling is applied.
5. **Multi-Layer Perceptron (MLP) Regression**: a feed-forward neural network trained with early stopping. We tune hidden

layer sizes $\in \{(64), (128), (64, 32)\}$, $\alpha \in \{0.0001, 0.001\}$, and use a learning rate of 0.001 with standard scaling.

All models are trained to predict `log_price`. Predictions are transformed back to the original price scale using $\exp(x) - 1$ when reporting price-level errors.

3.9 Evaluation Metrics

All models are evaluated using the same three metrics, reported both on the log-price scale and the original GBP price scale:

- **RMSE** (Root Mean Squared Error): the primary metric because it penalises large price errors more strongly.
- **MAE** (Mean Absolute Error): interpretable as the average absolute price deviation in GBP.
- R^2 (Coefficient of Determination): the proportion of target variance explained by the model.

For the preprocessing comparison, Ridge Regression and KNN are additionally evaluated under no scaling, standard scaling, and min–max scaling. This directly tests the effect of feature scaling on models whose optimisation or prediction depends on feature magnitudes.

4 Results

This section reports the model evaluation and optimisation results using the shared train–development–test split generated in Notebook 02. Model selection is based on the development set only; the held-out test set remains reserved for final evaluation. Since all models predict `log_price`, the reported price-scale metrics are computed after applying the inverse transformation to recover predicted prices in GBP.

4.1 Overall Model Comparison

Table 4 compares the five trained models on the development set. Random Forest achieves the best overall performance, with the lowest RMSE (GBP 1,866.08), the lowest MAE (GBP 1,217.09), and the highest price-scale R^2 (0.9541). This indicates that the ensemble model captures nonlinear relationships between price and features such as transmission, year, engine size, mileage, and brand/model effects more effectively than the simpler baselines.

KNN is the second-best model on the development set, with RMSE of GBP 1,987.53 and R^2 of 0.9479. However, its training performance is almost perfect (training $R^2 = 0.9995$), suggesting substantial overfitting. MLP also performs competitively but does not improve over Random Forest or KNN. Decision Tree and Ridge Regression provide useful baselines, but their higher RMSE values show the limitation of single-tree and linear assumptions for this prediction task.

Model	MAE	RMSE	R^2
Random Forest	1217.09	1866.08	0.9541
KNN Regression	1286.03	1987.53	0.9479
MLP Neural Network	1412.49	2129.92	0.9402
Decision Tree	1433.38	2192.47	0.9366
Ridge Regression	1671.34	2444.19	0.9213

Table 4: Development set performance on the original price scale. Lower MAE/RMSE and higher R^2 indicate better performance.

Table 5 reports the final held-out test performance after model selection. The ranking remains consistent with the development results, confirming that the selected models generalise well beyond the tuning split. Random Forest again achieves the best performance, with a test RMSE of GBP 1,793.69 and a price-scale R^2 of 0.9570. KNN remains the second-best model, while MLP, Decision Tree, and Ridge Regression follow the same relative order as on the development set.

Model	MAE	RMSE	R^2
Random Forest	1205.22	1793.69	0.9570
KNN Regression	1274.86	1924.35	0.9505
MLP Neural Network	1410.22	2058.39	0.9433
Decision Tree	1421.53	2130.05	0.9393
Ridge Regression	1671.39	2410.53	0.9223

Table 5: Held-out test set performance on the original price scale.

4.2 Hyperparameter Optimisation

The best hyperparameter settings selected during tuning are shown in Table 6. The selected Random Forest uses 200 trees and no fixed maximum depth, allowing the ensemble to model complex interactions while reducing variance through bagging. For KNN, distance weighting with $k = 10$ performs best, meaning closer neighbours contribute more strongly to the predicted price. Ridge Regression selects a small regularisation strength ($\alpha = 0.1$),

while the MLP selects a two-layer architecture with 64 and 32 hidden units.

Model	Scaling	Best settings
Ridge Regression	Standard	$\alpha = 0.1$
KNN Regression	Standard	$k = 10$, distance weights
Decision Tree	None	`min_samples_leaf=10`
Random Forest	None	200 trees, `min_samples_leaf=1`
MLP Neural Network	Standard	hidden layers $(64, 32)$, $\alpha = 0.0001$

Table 6: Best hyperparameters selected for each model family.

To check whether the main model-comparison pattern is stable across different training folds, we also run a supplementary 3-fold cross-validation experiment on a 12,000-row sample from the training split. This experiment is used as a robustness check only; final model selection is still based on the development set and final reporting on the held-out test set.

Model	CV RMSE	CV R^2
Random Forest	2223.83 ± 77.55	0.9323
KNN Regression	2331.47 ± 39.54	0.9256
Ridge Regression	2480.74 ± 65.66	0.9158
Decision Tree	2870.28 ± 37.95	0.8872
MLP Neural Network	4266.30 ± 2368.32	0.6779

Table 7: Supplementary 3-fold cross-validation results on a 12,000-row training subset. RMSE is reported on the original price scale as mean $\pm$ standard deviation.

The cross-validation check supports the main conclusion for the strongest models: Random Forest and KNN remain the two best-performing methods. The lower-ranked models are less stable across folds, especially MLP, whose large RMSE standard deviation indicates sensitivity to the sampled training data.

The optimisation results show a clear bias–variance pattern. Ridge Regression is stable but underfits because a linear function cannot fully represent the relationship between vehicle attributes and price. KNN has very low training error but a large train–development gap, showing high variance. Random Forest provides the best balance: its training error is lower than its development error, but the gap is much smaller than KNN, and its development and held-out test performance are the strongest overall.

4.3 Effect of Feature Scaling

Feature scaling has a strong effect on distance-based and linear models. Table 8 shows that Ridge Regression performs poorly without scaling (RMSE GBP 7,597.09), because features such as mileage and tax operate on much larger numeric scales than variables such as engine size. Standardisation and min–max scaling both substantially improve Ridge, reducing RMSE to approximately GBP 2,440.

KNN also benefits strongly from scaling. Without scaling, its development-tuning RMSE is GBP 6,889.90; after standardisation, this falls to GBP 2,097.92. Standard scaling outperforms min–max scaling for KNN in this experiment, likely because standardisation better preserves relative distances for skewed numerical variables.

Model	Scaling	RMSE	R^2
Ridge	None	7597.09	0.2393
Ridge	Standard	2444.19	0.9213
Ridge	Min–max	2442.51	0.9214
KNN	None	6889.90	0.3682
KNN	Standard	2097.92	0.9414
KNN	Min–max	2415.64	0.9223

Table 8: Scaling comparison for Ridge and KNN. KNN values are computed on the tuning subset for runtime efficiency.

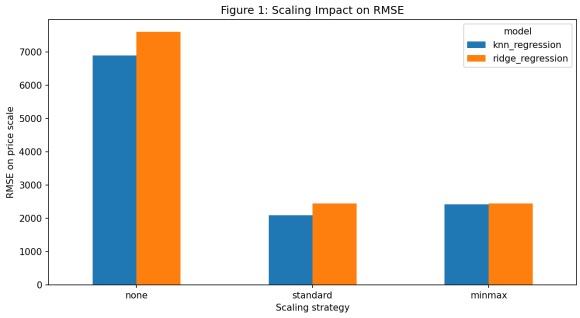

Figure 1: Effect of scaling strategy on RMSE for Ridge Regression and KNN.

4.4 Feature Influence

The Random Forest feature importance ranking provides insight into which attributes drive the strongest predictive signal. The most important feature is `transmission_Manual`, followed by `year`, `engineSize`, and `car_age`. This ranking is consistent with used-car pricing intuition: newer cars, larger engines, and transmission type are strongly associated with market value. Mileage, MPG, and the luxury-brand indicator also contribute meaningful signal, although their individual importances are smaller than the top structural vehicle features.

Ridge coefficients provide a complementary linear interpretation. The largest absolute coefficients are mostly model-specific indicators, such as `Caravelle`, `Up`, `KA`, `Aygo`, and `Supra`. Positive coefficients indicate models associated with higher log-price after controlling for other variables, while negative coefficients indicate lower-price models. Because these coefficients are model-based associations, they should be interpreted as predictive signals rather than causal effects.

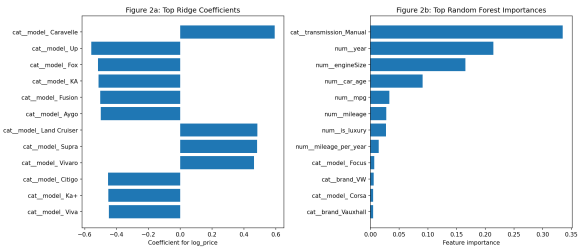

Figure 2: Top Ridge coefficients and Random Forest feature importances.

Overall, the evaluation supports the core research question: vehicle attributes can predict used-car prices with high accuracy. The best model, Random Forest, explains approximately 95.7% of the held-out test-set variance on the price scale, while maintaining a lower error than all baseline and neural models.

5 Discussion

[Discussion to be completed after results analysis.]

6 Conclusion

[Conclusion to be completed after all sections are written.]

Generative AI Usage

Claude (Anthropic) was used to assist with: (1) brainstorming research questions during the project planning phase; and (2) suggesting the initial project directory structure. No generative AI tools were used to write any part of this report, generate any dataset, or produce any experimental results. All analysis, code, and writing were produced by the authors.

References

Leo Breiman. 2001. Random forests. *Machine Learning*, 45(1):5–32.

Aditya Desai. 2021. UK Used Car Dataset. https://www.kaggle.com/datasets/adityadesai13/used-car-dataset-ford-and-mercedes. Accessed: May 2026.

Enis Gegic, Bakir Isakovic, Dino Keco, Zerina Masetic, and Jasmin Kevric. 2019. Car price prediction using machine learning techniques. In *TEM Journal*, volume 8, pages 113–118.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. Deep learning.

Mandalina Listiani. 2009. Support vector regression analysis for price predictions in the german car market. *Thesis, Hamburg University*.

Sameerchand Pudaruth. 2014. Predicting the price of used cars using machine learning techniques. *International Journal of Information & Computation Technology*, 4(7):753–764.

Sneha Shinde and Ketan Kotecha. 2018. Used car price prediction using K-Nearest Neighbor based model. *International Journal of Innovative Research in Computer and Communication Engineering*, 6(5).